

Cost of validating the ScottDomains development

What it costs, in wall-clock time and memory, to have Lean check this development. Every number here was produced by `scripts/compile.sh`, which wraps `lake build` in `GNU /usr/bin/time -v` and writes a log under `ScottDomains/logs/`; each row names the log it came from, so the measurement can be re-read rather than trusted.

Machine

#	Quantity	Value
1	CPU	12th Gen Intel Core i7-12700H, 20 logical cores
2	RAM	31.0 GiB (MemTotal 32,548,396 kB)
3	Lean	leanprover/lean4:v4.32.2
4	Mathlib on disk	7.4 GiB in <code>.lake/packages</code> , fetched prebuilt by <code>lake exe cache get</code>
5	Our build output	19 MiB in <code>.lake/build</code>
6	Development size	27 modules, 4440 lines, 199 theorems, 1 sorry

Mathlib is **not** compiled here. `lake exe cache get` downloads its `.olean` files, so every figure below is the cost of elaborating *our* 27 modules against a prebuilt Mathlib. Compiling Mathlib from source is hours of CPU and is not part of any measurement in this document.

The three costs worth distinguishing

#	What	Wall clock	Memory (PSS)	Log
1	Cached replay — nothing changed	0.72 s	769 MiB	<code>compile-20260806-190950.orchestrator</code>
2	One module re-elaborated (Skeleton/Lemma17, 452 lines)	1.50 s	1748 MiB	<code>compile-20260806-191132.orchestrator</code>
3	Whole library from scratch, Mathlib prebuilt	7.43 s	2275 MiB	<code>compile-20260806-190929.orchestrator</code>

Row 1 is what a no-op `lake build` costs and is the number not to quote: it measures trace-checking, not proof-checking. Row 3 is the honest answer to “what does validating the model cost”.

A note on how row 2 was produced: `touch` does not force re-elaboration, because Lake traces content hashes rather than mtimes. The artifacts (`.olean`, `.ilean`, `.trace`) must be removed.

Work and span

Summing the per-module times Lake reports in the row-3 log:

#	Quantity	Value
1	Work — sum of per-module elaboration	21.00 s over 25 modules
2	Span — wall clock for the whole build	7.43 s
3	Parallelism achieved (work / span)	2.83×

#	Quantity	Value
4	Cores available	20

The build uses about 3 of 20 cores. It is **dependency-bound, not core-bound**: the module import graph is a narrow chain, so more cores would not shorten it. The consequence for planning is that the way to check this development faster is to shorten the import chain, and the way to use the machine harder is to run independent builds concurrently — which is exactly what the five agent worktrees do.

Per-module cost is flat, between 0.8 s and 1.0 s for the twelve most expensive:

#	Module	Time
1	ScottHom	975 ms
2	StepFunction	963 ms
3	Smash	950 ms
4	Currying	950 ms
5	Projection	948 ms
6	Skeleton/Lemma17	946 ms
7	StrictHom	944 ms
8	NormalSubposet	929 ms
9	CoalescedSum	908 ms
10	Lift	870 ms
11	EffectivePresentation	858 ms
12	Powerset	855 ms

That flatness says the cost is dominated by loading Mathlib's .olean into each worker, not by the difficulty of any one proof. No module in this development is a hot spot.

Memory constraint, and why one number is not enough

Three figures are recorded per run, because the obvious one is wrong in both directions. For the whole-library build:

#	Figure	Value	What it is	Error
1	GNU time -v max RSS	1674 MiB	peak of the largest single process in the tree	understates — ignores that 20 Lean workers run at once
2	Σ RSS over the process group	11774 MiB	every process's resident pages added up	overstates by ~5× — Mathlib's .olean are mmaped read-only and shared, and each sharer is charged in full
3	Σ PSS over the process group	2275 MiB	shared pages apportioned across the processes mapping them	the true footprint

PSS is the figure to size RAM against: **a whole validation needs about 2.3 GiB**. Quoting 1.7 GiB would under-provision a parallel build; quoting 11.8 GiB would over-provision it by a factor of five and wrongly rule out running five agents on this machine.

For the five-worktree layout of round r0028:

#	Resource	Per worktree	Five worktrees	Against
1	Memory (PSS)	2.3 GiB	≤ 11.4 GiB	31 GiB RAM
2	Checkout on disk	327 MiB	1.6 GiB	—
3	Mathlib	0 — symlinked	7.4 GiB once	vs. 37 GiB if each vendored its own

The 11.4 GiB is an upper bound and the real figure is lower: every worktree symlinks the *same* `.lake/packages`, so the `.olean` pages are shared across agents as well as within one build. Memory is not the binding constraint at five agents, and neither is disk.

How parallel can we go?

Measured directly: N checkouts building the whole library at once, each with its own `.lake/build` and all sharing `.lake/packages` by symlink — the r0028 agent layout exactly.

#	Concurrent builds	Span	Throughput (builds/s)	Speedup vs. serial	Load average
1	1	10.53 s	0.095	1.00×	4.7
2	2	13.71 s	0.146	1.54×	5.6
3	3	17.43 s	0.172	1.81×	8.9
4	6	30.16 s	0.199	2.10×	18.2

Throughput saturates fast. Going from 3 to 6 concurrent builds buys **10% more throughput** while inflating each build’s latency from 17 s to 30 s. The reason is in the work/span figures above: one build already draws ~ 2.8 cores, so six of them demand ~ 17 — and the load average of 18.2 confirms the machine was saturated.

Under a 12-core budget, the ceiling is 4 concurrent builds ($4 \times 2.8 \approx 11.2$).

But concurrent *builds* is not the same as concurrent *agents*, and that is the number that matters. An agent spends most of its wall time reasoning and reading, not building. During r0028’s five-agent round the load average sampled 0.56, 0.68, 0.81, 0.94 and 1.44 against 20 cores — five agents together drew about **1.2 cores**, roughly 0.25 each, a duty cycle near 9%. Extrapolating, a 12-core budget is not reached until far more agents than the collision and review limits below allow.

#	Constraint	Binds at	Why
1	CPU, average	~ 48 agents	0.25 cores per agent against a 12-core budget
2	CPU, bursts	~ 8 agents	at a 9% duty cycle, ~ 4 build simultaneously, which is the 12-core ceiling
3	Memory	~ 5 concurrent builds	2.3 GiB PSS each against 12 GiB of a 31 GiB machine

#	Constraint	Binds at	Why
4	Declaration collisions	~6 agents	r0028 produced 2 name clashes among 5 agents (10 pairs). Pairs grow as $N(N-1)/2$, so 8 agents would be ~5 clashes
5	Review bandwidth	~5 agents	r0028 was ~3,800 lines of Lean to read, merge and audit in one round

Recommendation: 4 to 6 agents. Constraint 4 is the real limit, not the hardware — and unlike the hardware it is fixable: give each agent its own namespace (`ScottDomains.Hoare`, `ScottDomains.Smyth`, ...) so two agents cannot mint the same fully-qualified name. That was adopted in r0029.

`LEAN_NUM_THREADS` is the throttle if bursts need capping: at $N=3$ it cut load average from 8.9 to 6.9 with the span unchanged (17.4 s $\rightarrow$ 17.1 s). Lake has no `-j` flag — `lake build -j 3` fails with unknown short option `'-j'`.

Method, and how to repeat it

```
scripts/compile.sh [-r rNNNN] [lake target ...]
```

The wrapper detects its GRASE role from the checkout path (`*-agentN` $\Rightarrow$ `agentN`, otherwise orchestrator), strips ANSI escapes, tags the log with the round, and appends a `--- times ---` footer whose two field names are verbatim from `~/projects/GRASE/standards/LoggingStandard.md` so one parser reads every project's logs. It exits with `lake build`'s own status.

Memory comes from a sampler that reads `/proc/<pid>/smaps_rollop` every 200 ms for each process in the build's own process group, summing `Rss` and `Pss` separately and keeping both maxima. `setsid` puts the build in its own process group, so a measurement taken while the agent worktrees are building does not count their processes. A build shorter than one 200 ms interval reports the tree figures as not sampled rather than as zero.

Error counting distinguishes **Lean diagnostics**, which carry a source position (`error: File.lean:12:0: ...`), from **Lake's summary lines** (`error: build failed`), which do not. Counting them together inflated two bad theorems into six errors. The counts are taken from the log itself, so they describe exactly what was recorded.

Both behaviours were tested against a deliberately broken module: four diagnostics from two bad theorems, two Lake error lines, exit status 1 propagated through the wrapper, and a complete log written.

To reproduce row 3:

```
rm -rf ScottDomains/.lake/build/lib/lean/ScottDomains*
scripts/compile.sh -r rNNNN
```

Measurement conditions

Rows 1–3 were taken while the five r0028 agents were running in their own worktrees. Load average was 0.94 before the row-3 build, against 20 cores, so the machine was near-idle and the figures are not badly contended — but they are not quiet-machine figures either. A repeat measurement on an idle machine is worth taking before these numbers are used as a baseline for regression.

An earlier revision of this document reported 795 MiB, 1674 MiB and 1674 MiB for the three rows. Those were GNU time -v's single-process maxima, taken before the process-group sampler existed; they understated the whole-library build by about 600 MiB. They are superseded by the PSS column above.